

APE 007: Integración de la aplicación móvil con el Backend desarrollado

Integrantes:

- Eberson Guayllas
- Wilmer Pardo
- Santiago Tamayo

Fecha:

01/03/2026

I. ENPOINTS CONSUMIDOS

Para la aplicación móvil que estamos construyendo se realizarán las consultas a los Endpoints que se construyeron en las unidades anteriores, el backend en cuestión se encarga de ofrecer los servicios necesarios para que los usuarios resuelvan, analicen, comenten e intercambien ideas sobre distintos retos de programación, estos retos se presentan en tres categorías: Fácil, medio y difícil.

Las distintas rutas de esta API REST se encuentran clasificadas de la siguiente manera:

- Rutas para el módulo de personas: Este módulo gestiona la información de toda la actividad producida por los usuarios, con esto hacemos referencia al registro de sus datos, modificación de sus perfiles y para el administrador el listado de todos estos. Los endpoints encargados de este trabajo están especificados en las figuras 1, 2, y 3

```
14  ## -----
15  ## RUTA GET ALL (con paginación)
16  ## -----
17  @persona_bp.route('/personas', methods=['GET'])
18  def get_personas():
19      try:
20          page = request.args.get('page', 1, type=int)
21          per_page = request.args.get('per_page', 20, type=int)
22          personas = PersonaModel.get_all_persons(page, per_page)
23          return jsonify(personas), 200
24      except Exception as e:
25          print(f"Error en get_personas: {e}")
26          return jsonify({"error": "Error interno del servidor"}), 500
27
28
29  ## -----
30  ## RUTA GET BY ID
31  ## -----
32  @persona_bp.route('/personas/<int:id_persona>', methods=['GET'])
33  def get_person(id_persona):
34      try:
35          persona = PersonaModel.get_persona_by_id(id_persona)
36          if persona:
37              return jsonify(persona), 200
38          else:
39              return jsonify({"error": "Persona no encontrada"}), 404
40      except Exception as e:
41          print(f"Error en get_person: {e}")
42          return jsonify({"error": "Error interno del servidor"}), 500
43
```

Fig 1. Rutas para obtener todas o una persona específica

```

45  ## -----
46  ## RUTA UPDATE (PUT)
47  ## -----
48  @person_bp.route('/personas/<int:id_persona>', methods=['PUT'])
49  def update_person(id_persona):
50
51      data = request.get_json()
52      if not data:
53          return jsonify({"error": "No se proporcionaron datos para actualizar"}), 400
54
55      try:
56          # <<< 2. CORRECCIÓN: El modelo devuelve un dict, el controlador lo "jsonifica" >>>
57          response_dict, status_code = PersonaModel.update_person(id_persona, data)
58          return jsonify(response_dict), status_code
59
60      except Exception as e:
61          print(f"Error en update_person: {e}")
62          return jsonify({"error": "Error interno del servidor"}), 500
63  # -----
64
65  @person_bp.route('/personas/<int:id_persona>/password', methods=['PUT'])
66  def change_password(id_persona):
67      data = request.get_json()
68
69      if not data or 'password' not in data:
70          return jsonify({"error": "Contraseña requerida"}), 400
71
72      password_hash = generate_password_hash(data['password'])
73
74      response, status = PersonaModel.update_password(id_persona, password_hash)
75      return jsonify(response), status
76

```

Fig 2. Rutas para actualizar una persona o cambiar su contraseña.

```

115  # -----
116  # GET Ranking
117  # -----
118  @person_bp.route('/ranking', methods=['GET'])
119  @jwt_required()
120  def get_ranking_leaderboard():
121
122      # Validar que el usuario del token existe
123      try:
124          id_persona_str = get_jwt_identity()
125          id_persona_actual = int(id_persona_str)
126      except (ValueError, TypeError):
127          return jsonify({"error": "Token inválido"}), 422
128
129      if not PersonaModel.get_persona_by_id(id_persona_actual):
130          return jsonify({"error": "Usuario del token no encontrado"}), 401
131
132      try:
133          page = request.args.get('page', 1, type=int)
134          per_page = request.args.get('per_page', 10, type=int)
135
136          ranking_data = PersonaModel.get_ranking(page, per_page)
137
138          return jsonify(ranking_data), 200
139
140      except Exception as e:
141          print(f"Error en GET /ranking: {e}")
142          return jsonify({"error": "Error interno del servidor"}), 500
143
144

```

Fig 3. Ruta para obtener el ranking de una persona en específico.

- Rutas para el módulo de interacción social: Las rutas destinadas a este módulo son más simples, pero no menos importantes, aquí se encuentran las entradas a la api que permiten a los usuarios generar publicaciones acerca de dudas o experiencias que han tenido resolviendo los retos, así como solicitar ayuda a la red de usuarios, estos podrán reaccionar o comentar las mismas para otorgar dinamismo a la aplicación. Las rutas destinadas a cumplir con los servicios especificados se muestran en las figuras 4,5 y 6.

```

20 @publication_bp.route('/', methods=['POST'])
21 @jwt_required()
22 def crear_publicacion():
23     id_persona, error_response = get_current_user_id()
24     if error_response: return error_response
25
26     data = request.json
27     contenido = data.get('contenido')
28     if not contenido:
29         return jsonify({"error": "El 'contenido' es requerido"}), 400
30
31     try:
32         response, status = PublicacionModel.create_post(id_persona, contenido)
33         return jsonify(response), status
34     except Exception as e:
35         return jsonify({"error": str(e)}), 500
36
37
38
39 @publication_bp.route('/', methods=['GET'])
40 @jwt_required() # --- AQUÍ ES OBLIGATORIO ESTAR LOGUEADO PARA VER EL FEED CORRECTAMENTE
41 def obtener_feed():
42     # 1. Diferenciamos quién está mirando
43     try:
44         id_persona_vision = int(get_jwt_identity())
45     except:
46         return jsonify({"error": "Token inválido"}), 422
47
48     try:
49         page = request.args.get('page', 1, type=int)
50
51         # 2. Pasamos ese ID al modelo nuevo
52         posts = PublicacionModel.get_all_posts(id_persona_vision, page=page)
53
54         return jsonify(posts), 200
55     except Exception as e:
56         print(e)
57         return jsonify({"error": "Error interno"}), 500
58

```

Fig 4. EndPoints para crear publicación y obtener el feed completo

```

61 @publication_bp.route('/<int:id_publicacion>/comentarios', methods=['POST'])
62 @jwt_required()
63 def crear_comentario(id_publicacion):
64     id_persona, error_response = get_current_user_id()
65     if error_response: return error_response
66
67     data = request.json
68     contenido = data.get('contenido')
69     id_comentario_padre = data.get('id_comentario_padre') # Opcional
70     if not contenido:
71         return jsonify({"error": "El 'contenido' es requerido"}), 400
72
73     try:
74         response, status = PublicacionModel.create_comment(
75             id_persona, id_publicacion, contenido, id_comentario_padre
76         )
77         return jsonify(response), status
78     except Exception as e:
79         return jsonify({"error": str(e)}), 500
80
81 # --- RUTAS DE REACCIONES ---

```

Fig 5. EndPoints encargados para crear un comentario enlazado a una publicación en específico.

```

83 @publication_bp.route('/<int:id_publicacion>/reacciones', methods=['POST'])
84 @jwt_required()
85 def reaccionar_publicacion(id_publicacion):
86     id_persona, error_response = get_current_user_id()
87     if error_response: return error_response
88
89     data = request.json
90     id_tipo_reaccion = data.get('id_tipo_reaccion')
91     if not id_tipo_reaccion:
92         return jsonify({"error": "El 'id_tipo_reaccion' es requerido"}), 400
93
94     try:
95         response, status = PublicacionModel.set_reaction(
96             id_persona, id_publicacion, id_tipo_reaccion
97         )
98         return jsonify(response), status
99     except Exception as e:
100         return jsonify({"error": str(e)}), 500
101

```

Fig 6. EndPoints encargados de permitir las reacciones a las publicaciones.

- Rutas para el módulo de retos: Este posiblemente es uno de los módulos más importantes de la aplicación. Ya que, si bien es cierto que la aplicación móvil no permite la resolución de los retos de manera directa, la versión web si lo hace, y para esto el usuario debe estar informado en todo momento sobre los lenguajes en que el reto puede ser resuelto y claramente sobre que trata dicho reto. Las rutas para este módulo se describen las figuras 7,8 y 9.

```

16 # Retos_bp.route('/', methods=['POST'])
17 @jwt_required()
18 def crear_nuevo_reto():
19     try:
20         id_persona_str = get_jwt_identity()
21         id_persona_actual = int(id_persona_str) # Convertir de string a int
22     except (ValueError, TypeError):
23         return jsonify({"error": "Token inválido (identidad no numérica)"}), 422 # 422 Unprocessable Entity
24
25     persona = PersonaModel.get_persona_by_id(id_persona_actual)
26     if not persona:
27         return jsonify({"error": "Usuario del token no encontrado"}), 401 # 401 Unauthorized
28     # --- CORRECCIÓN INICIA ---
29
30     # (Opcional) Verificar rol de Admin/tutor
31     # if persona['id_rol'] not in [1, 3]: # 1:Admin, 3:tutor
32     #     return jsonify({"error": "No tienes permisos para crear retos"}), 403
33
34     data = request.json
35     if not data:
36         return jsonify({"error": "No se recibió ningún dato"}), 400
37
38     required_keys = ['titulo', 'descripcion', 'nombre_dificultad', 'lenguajes', 'tests']
39     if not all(k in data for k in required_keys):
40         return jsonify({"error": "Faltan datos requeridos (titulo, descripcion, nombre_dificultad, lenguajes, tests)"}), 400
41
42     try:
43         response, status_code = RetosModel.create_reto(data)
44         return jsonify(response), status_code
45     except Exception as e:
46         print(f"Error en retoscontroller POST: {e}")
47         return jsonify({"error": "Error interno del servidor", "detalle": str(e)}), 500

```

Fig 7. Ruta destinada para la creación de nuevos retos (solo para el administrador)

```

71 @retos_bp.route('/<int:id_reto>', methods=['GET'])
72 @jwt_required()
73 def get_reto(id_reto):
74     # --- CORRECCIÓN INICIA ---
75     # Validar que el usuario del token existe y es un entero válido
76     try:
77         id_persona_str = get_jwt_identity()
78         id_persona_actual = int(id_persona_str) # Convertir de string a int
79     except (ValueError, TypeError):
80         return jsonify({"error": "Token inválido (identidad no numérica)"}), 422
81
82     if not PersonaModel.get_persona_by_id(id_persona_actual):
83         return jsonify({"error": "Usuario del token no encontrado"}), 401
84     # --- CORRECCIÓN TERMINA ---
85
86     try:
87         # Llamar al modelo para obtener el detalle del reto
88         reto_detalle = RetosModel.get_reto_by_id(id_reto)
89
90         if reto_detalle:
91             return jsonify(reto_detalle), 200
92         else:
93             return jsonify({"error": "Reto no encontrado"}), 404
94
95     except Exception as e:
96         print(f"Error en retoscontroller GET: {e}")
97         return jsonify({"error": "Error interno del servidor"}), 500

```

Fig 8. Ruta destinada a obtener un reto específico.

```

102 # --- RUTA POST para ENVIAR una respuesta a un reto ---
103 # --- CORRECCIÓN INICIA ---
104 @retos_bp.route('/<int:id_reto>/submit', methods=['POST'])
105 @jwt_required()
106 def submit_respuesta(id_reto):
107     # 1. Usuario autenticado
108     try:
109         id_persona = int(get_jwt_identity())
110     except:
111         return jsonify({"error": "Token inválido"}), 422
112
113     if not PersonaModel.get_persona_by_id(id_persona):
114         return jsonify({"error": "Usuario no encontrado"}), 401
115
116     # 2. JSON (FORZADO)
117     data = request.get_json(force=True)
118     print("DATA RECIBIDA:", data)
119
120     codigo_fuente = data.get("codigo_fuente")
121     id_lenguaje = data.get("id_lenguaje")
122
123     if codigo_fuente is None or id_lenguaje is None:
124         return jsonify({"error": "Faltan 'codigo_fuente' o 'id_lenguaje'", "data": data}), 400
125
126     if not isinstance(id_lenguaje, int):
127         return jsonify({"error": "id_lenguaje debe ser INT"}), 400
128
129     # 3. Guardar respuesta
130     response, status = RespuestaModel.create_submission(
131         id_persona=id_persona,

```

Fig 9. Ruta destinada a enviar una respuesta para un reto específico

II. EJEMPLOS DE SOLICITUD DE RESPUESTA

Para los ejemplos de peticiones de solicitud podemos basarnos en los distintos tipos de solicitud HTTP que existen. Por ejemplo, si mandamos una solicitud PUT para registrar un nuevo usuario dentro de nuestra aplicación en la consola de solida obtendremos un código 201 que indica un registro de usuario exitoso.

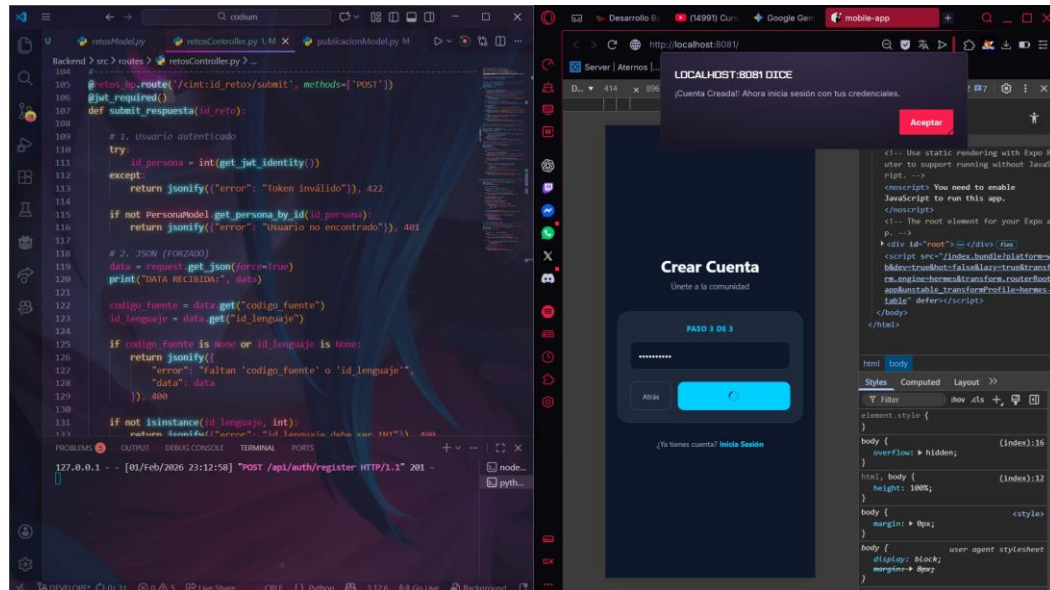


Fig 10. Registro de usuario y respuesta del servidor exitosa

La figura 10 podemos ver como al ingresar todos los campos de registro con valores correctos (sin dejar campos vacíos) se puede registrar un nuevo usuario en la base de datos

III. CAPTURAS DE PANTALLA DE LA APP EN EJECUCIÓN

Al utilizar react-native se pueden construir aplicaciones multiplataforma, en esta unidad se nos pide realizar una aplicación que funcione en la plataforma móvil por tanto estamos construyendo una aplicación multiplataforma IOS/ANDROID por ello se precisa de un emulador virtual o de conectar un dispositivo móvil a través de cable de datos.

